

CPUFlow

CPU Scheduling & Architecture Performance Simulator

Beginner User Guide / Project Guide Book

This guide explains what to run, what to enter, when to enter it, what each option does, and how information moves from one part of CPUFlow to the next. It is written so that a new user can understand and demonstrate the project without already knowing the complete code.

1. What is CPUFlow?

CPUFlow is a C-based simulator that connects Data Structures and Algorithms (DSA) concepts with Computer Organization and Architecture (COA). It models processes, a ready queue, CPU scheduling, cache and memory access, context switching, and performance analysis.

Main idea: the project shows how processes move toward the CPU and how scheduling, memory/cache behavior, and context switching contribute to overall CPU performance.

2. Project Parts

Part	Module	Main Purpose
1	Process Management	Create, display, search, update and delete processes
2	Ready Queue	Select processes that are ready to use the CPU
3	CPU Scheduling	Apply FCFS, SJF, SRTF, Priority and Round Robin
4	CPU Architecture	Represent CPU-side architecture concepts
5	Cache & Memory	Store memory values and simulate cache hits/misses
6	Context Switching	Save and load process CPU state
7	Performance Analysis	Calculate CPU and scheduling performance
8	Final Integration	Use the complete CPUFlow simulator in one program

3. Correct Order of Execution

The most important thing to understand is that the parts are connected through files. Therefore, use them in this order:

```
Part 1
↓
processes.txt
↓
Part 2
↓
readyQueue.txt
↓
Part 3 ■■■■■■■■→ schedulingResults.txt
↓
Part 5 ■■■■■■■■→ cacheResults.txt
↓
Part 6 ■■■■■■■■→ contextSwitches.txt
↓
Part 7
↓
Performance Analysis
```

Important: Part 6 does not ask for the number of memory locations. If you see that question, you are running Part 5 (Cache & Memory).

4. Part 1 — Process Management

Purpose: create the processes that will be used by the rest of the simulator.

Menu

1. Create Process
2. Display Process
3. Search Process
4. Update Process
5. Delete Process
6. Exit

What do the process values mean?

Input	Meaning	Example
Process ID	Unique process number	1
Arrival Time	Time when process enters the system	0
Burst Time	CPU time required by the process	5
Priority	Priority value used by priority scheduling	2

Recommended first input

Choice: 1

Number of Process: 4

Process 1

ID: 1

Arrival Time: 0

Burst Time: 5

Priority: 2

Process 2

ID: 2

Arrival Time: 1

Burst Time: 3

Priority: 1

Process 3

ID: 3

Arrival Time: 2

Burst Time: 4

Priority: 3

Process 4

ID: 4

Arrival Time: 3

Burst Time: 2

Priority: 2

After creation, CPUFlow saves these records in **processes.txt**. That file becomes the input for Part 2.

5. Part 2 — Ready Queue

Purpose: choose which created processes are currently ready to compete for the CPU. Part 2 reads processes.txt generated by Part 1.

1. Add Process to Queue
2. Remove Process from Queue
3. Display Ready Queue
4. Peek
5. Exit

For the recommended demonstration, add P1, P2, P3 and P4.

Choice: 1
Process ID: 1

Choice: 1
Process ID: 2

Choice: 1
Process ID: 3

Choice: 1
Process ID: 4

Choice: 3

The selected processes are saved in **readyQueue.txt**. Part 3 reads this file and schedules only these ready processes.

6. Part 3 — CPU Scheduling

Purpose: decide which ready process gets the CPU and in what order.

1. FCFS
2. SJF
3. SRTF
4. Priority Scheduling
5. Round Robin
6. Exit

Recommended demonstration: choose SRTF.

Choice: 3
(wait for the scheduling result)
Choice: 6

The program calculates Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT). The formulas are:

Turnaround Time = Completion Time - Arrival Time
Waiting Time = Turnaround Time - Burst Time

The latest scheduling result is saved in **schedulingResults.txt**. Part 7 uses this result for performance analysis.

7. Part 4 — CPU Architecture

This section represents the architecture side of the project. It is intended to connect the scheduling activity with CPU-related concepts such as the processor, execution state, memory, cache and context switching.

If your current Part 4 is a separate program, run it according to its own menu. The integrated Part 8 combines the CPU architecture-related behavior with the other modules.

8. Part 5 — Cache & Memory

Purpose: simulate main memory and a small cache. You first create memory values, then access addresses. If an address is already in the cache, it produces a cache hit; otherwise it produces a miss.

1. Display Main Memory
2. Display Cache
3. Access Memory
4. Display Cache Statistics
5. Reset Cache
6. Exit

First enter the number of memory locations. For example:

Number of memory locations: 5

Enter memory values:

10
20
30
40
50

Now use Access Memory. Accessing the same address twice is useful for demonstrating a hit.

Choice: 3
Address: 2
→ Cache Miss

Choice: 3
Address: 2
→ Cache Hit

Choice: 4
→ Displays hits, misses and hit rate

The cache statistics are saved in **cacheResults.txt** for Part 7.

9. Part 6 — Context Switching

Purpose: demonstrate how the CPU saves the state of one process and loads the state of another.

1. Display Processes
2. Run Process
3. Execute Process
4. Context Switch
5. Display CPU State
6. Reset Process State
7. Exit

Recommended demonstration

Choice: 2
Process ID: 1

Choice: 3
Instructions: 5

Choice: 4
Switch to Process ID: 2

Choice: 3

Instructions: 3

Choice: 4

Switch to Process ID: 3

Choice: 5

When switching from P1 to P2, the current PC and accumulator of P1 are retained and the saved state of P2 is loaded. The context-switch counter increases by one.

The total number of context switches is saved in **contextSwitches.txt**.

10. Part 7 — Performance Analysis

Purpose: automatically collect the results produced by scheduling, cache and context-switching modules and calculate overall performance. No performance values need to be typed manually.

1. Calculate Performance
2. Display Scheduling Results
3. Display Cache Results
4. Display Context Switch Results
5. Exit

Recommended input:

Choice: 1

The program reads schedulingResults.txt, cacheResults.txt and contextSwitches.txt. Typical results include average waiting time, average turnaround time, CPU utilization, throughput, cache hit/miss information and total context switches.

11. Part 8 — Final Integrated CPUFlow Simulator

Part 8 is the final version. Instead of opening eight separate programs, it provides one main menu from which the major modules can be accessed.

```
=====
                        CPUFLOW SIMULATOR
=====
1. Process Management
2. Ready Queue
3. CPU Scheduling
4. Cache & Memory
5. Context Switching
6. Performance Analysis
7. Exit
=====
```

For a complete demonstration, start with Process Management, then Ready Queue, Scheduling, Cache & Memory, Context Switching and finally Performance Analysis.

12. Complete Demonstration Sequence

The following is a simple demonstration sequence. Menu numbers refer to the Part 8 integrated simulator.

```
MAIN MENU
1 → Process Management
    1 → Create Process
    Enter 4 processes
    6 → Exit module
```

2 → Ready Queue
 1 → Add P1
 1 → Add P2
 1 → Add P3
 1 → Add P4
 5 → Exit module

3 → CPU Scheduling
 3 → SRTF
 6 → Exit module

4 → Cache & Memory
 Enter 5 memory locations
 Enter: 10, 20, 30, 40, 50
 Access address 2 twice
 Display statistics
 6 → Exit module

5 → Context Switching
 Run P1
 Execute 5 instructions
 Switch to P2
 Execute 3 instructions
 Switch to P3
 Display CPU state
 7 → Exit module

6 → Performance Analysis
 1 → Calculate Performance

7 → Exit

13. Files Created by CPUFlow

File	Created/Used For
processes.txt	Stores process information from Part 1
readyQueue.txt	Stores processes selected for the ready queue
schedulingResults.txt	Stores the latest scheduling algorithm result
cacheResults.txt	Stores cache hit/miss statistics
contextSwitches.txt	Stores total context switches

14. Important Terms

Term	Simple Meaning
Process	A program/task waiting to use the CPU
Arrival Time	When a process becomes available
Burst Time	CPU time needed by a process
Ready Queue	Processes waiting for CPU execution
Scheduling	Selecting which process gets CPU time
Cache Hit	Requested data is already in cache
Cache Miss	Requested data is not in cache
Context Switch	Changing CPU execution from one process to another
PC	Program Counter; stores the current instruction position

Accumulator	Register-like value used here to represent CPU state
Throughput	Number of completed processes per unit time
CPU Utilization	Percentage of available CPU time spent executing

15. Troubleshooting

Problem: File not found

Run the previous module first. For example, Part 2 needs processes.txt, while Part 3 needs readyQueue.txt. Keep the generated files in the same folder as the program when using the separate modules.

Problem: Part 6 asks for memory locations

You are most likely running Part 5. Part 6 starts by reading readyQueue.txt and does not ask for memory locations.

Problem: Performance values are zero or missing

Make sure you have already run scheduling, cache/memory and context-switching modules so their result files contain data.

Problem: The latest scheduling algorithm is different from the one expected

schedulingResults.txt stores the latest scheduling result. Run the required algorithm again before Part 7.

16. Project Flow in One Sentence

Create processes → place selected processes in the ready queue → schedule them → simulate memory/cache → simulate context switching → calculate performance.

This is the main story of CPUFlow and can also be used when explaining the project during a viva or demonstration.